Binary Analysis

Paul Joo-Hyun Kim (pauljoohyunkim@gmail.com)

November 2, 2026

Binary Analysis

Binary Analysis

Techniques to understand what a binary executable does. Sometimes source code is given, often times not.

Warning

This is a vast topic. Obviously you will need to do your own research.

What you need

Tools and Resources

- Linux PC (Preferable)
- Specific Architectural Documentation / Datasheet
  - Although tools often decode this information for you.
- Debugger
  - GDB
- Binutils/util-linux/etc.
  - Especially `objdump`, `strings`, `hexdump`.
- Binary Analysis Tools

Binary Analysis Tools

Binary Analysis Tools

- IDA Pro
  - Industry standard disassembler/debugger
  - Commercial Use: \$1,099 $\sim$ \$6,899 per year.
- Hopper
  - Mac-native Analysis tool.
  - Personal License (\$99.00), Computer License (\$129.00)
- Radare2
  - FOSS. LGPLv3 License
  - No GUI
- Ghidra
  - FOSS. Apache 2.0
  - Developed by NSA

How to get Started

To-Do

- Determine the scope of analysis.
- Run the binary if possible.
- Check for symbol table or disassembly.
- Use a debugger to step through the instructions.
- Use binary analysis tools.
- Use side-channeling analysis techniques.
- ⋮ (Not an exhaustive list. No need for order.)

Scope of Analysis

Scope

- Do I only care about extracting sensitive information? (Keys, tokens, etc.)
- Do I care about the program's behavior?
- Do I care about causing unconventional bugs and misdirection of flow?

Demo Target

```c
// gcc -Wall password_check.c -o password_check
#include <string.h>
#include <stdio.h>

int checkKey(const char* input) {
    if (strcmp(input, "testpass") == 0) {
        printf("Unlocked\n");
        return 0;
    } else {
        printf("Wrong key\n");
        return 2;
    }
}

int main(int argc, char* argv[]) {
    if (argc == 1) {
        printf("Usage: prog.c key\n");
        return 1;
    }

    return checkKey(argv[1]);
}
```

Running the Binary

```
$ pbjk (demo) > make
gcc -Wall password_check.c -o password_check
$ pbjk (demo) > ./password_check; echo $?
Usage: prog.c key
1
$ pbjk (demo) > ./password_check garbage; echo $?
Wrong key
2
$ pbjk (demo) > ./password_check testpass; echo $?
Unlocked
0
```

Trivial Analysis

strings

Binutils includes `strings` for finding all strings (by default, of length 4 or more).

```
$ pbjk (demo) > strings password_check
/lib64/ld-linux-x86-64.so.2
puts
__libc_start_main
__cxa_finalize
strcmp
libc.so.6
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
testpass
Unlocked
Wrong key
Usage: prog.c key
9*3$"
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
```

Symbol Table and Disassembly

Binutils includes `objdump` for finding symbols and reading assembly instructions.

```
$ pbjk (demo) > objdump -t password_check

password_check:     file format elf64-x86-64

SYMBOL TABLE:
0000000000000000 l    df *ABS*  0000000000000000              Scrt1.o
000000000000038c l     O .note.ABI-tag  0000000000000020              __abi_tag
0000000000000000 l    df *ABS*  0000000000000000              crtstuff.c
00000000000010b0 l     F .text  0000000000000000              deregister_tm_clones
00000000000010e0 l     F .text  0000000000000000              register_tm_clones
0000000000001120 l     F .text  0000000000000000              __do_global_dtors_aux
0000000000004010 l     O .bss   0000000000000001              completed.0
0000000000003db8 l     O .fini_array    0000000000000000              __do_global_dtors_aux_fini_array_entry
0000000000001160 l     F .text  0000000000000000              frame_dummy
0000000000003db0 l     O .init_array    0000000000000000              __frame_dummy_init_array_entry
0000000000000000 l    df *ABS*  0000000000000000              password_check.c
0000000000000000 l    df *ABS*  0000000000000000              crtstuff.c
0000000000002138 l     O .eh_frame      0000000000000000              __FRAME_END__
0000000000000000 l    df *ABS*  0000000000000000
0000000000003dc0 l     O .dynamic       0000000000000000              _DYNAMIC
0000000000002034 l       .eh_frame_hdr  0000000000000000              __GNU_EH_FRAME_HDR
0000000000003fb0 l     O .got   0000000000000000              _GLOBAL_OFFSET_TABLE_
0000000000000000       F *UND*  0000000000000000              __libc_start_main@GLIBC_2.34
0000000000000000  w      *UND*  0000000000000000              _ITM_deregisterTMCloneTable
0000000000004000  w      .data  0000000000000000              data_start
0000000000000000       F *UND*  0000000000000000              puts@GLIBC_2.2.5
0000000000004010 g       .data  0000000000000000              _edata
0000000000001204 g     F .fini  0000000000000000              .hidden _fini
0000000000001169 g     F .text  0000000000000056              checkKey
0000000000004000 g       .data  0000000000000000              __data_start
```

```
0000000000001169 <checkKey>:
    1169:       f3 0f 1e fa             endbr64
    116d:       55                      push   %rbp
    116e:       48 89 e5                mov    %rsp,%rbp
    1171:       48 83 ec 10             sub    $0x10,%rsp
    1175:       48 89 7d f8             mov    %rdi,-0x8(%rbp)
    1179:       48 8b 45 f8             mov    -0x8(%rbp),%rax
    117d:       48 8d 15 80 0e 00 00    lea    0xe80(%rip),%rdx        # 2004 <_IO_stdin_used+0x4>
    1184:       48 89 d6                mov    %rdx,%rsi
    1187:       48 89 c7                mov    %rax,%rdi
    118a:       e8 e1 fe ff ff          call   1070 <strcmp@plt>
    118f:       85 c0                   test   %eax,%eax
    1191:       75 16                   jne    11a9 <checkKey+0x40>
    1193:       48 8d 05 73 0e 00 00    lea    0xe73(%rip),%rax        # 200d <_IO_stdin_used+0xd>
    119a:       48 89 c7                mov    %rax,%rdi
    119d:       e8 be fe ff ff          call   1060 <puts@plt>
    11a2:       b8 00 00 00 00          mov    $0x0,%eax
    11a7:       eb 14                   jmp    11bd <checkKey+0x54>
    11a9:       48 8d 05 66 0e 00 00    lea    0xe66(%rip),%rax        # 2016 <_IO_stdin_used+0x16>
    11b0:       48 89 c7                mov    %rax,%rdi
    11b3:       e8 a8 fe ff ff          call   1060 <puts@plt>
    11b8:       b8 02 00 00 00          mov    $0x2,%eax
    11bd:       c9                      leave
    11be:       c3                      ret

00000000000011bf <main>:
    11bf:       f3 0f 1e fa             endbr64
    11c3:       55                      push   %rbp
```

Stripping Symbol

Strip Symbol

To make binary analysis more annoying for attacker, optimize and strip all the symbols before production.

```
$ pbjk (demo) > strip password_check
$ pbjk (demo) > objdump -t password_check

password_check:     file format elf64-x86-64

SYMBOL TABLE:
no symbols


$ pbjk (demo) >
```

Warning

This does not mean it is safe.

Using Debugger

GDB

Can be used to analyze/redirect flow at runtime.

- In the following demo, put breakpoint at checkKey, and run with `r notthepassword`


Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Breakpoint 1, 0x0000555555555171 in checkKey ()
(gdb) disass
Dump of assembler code for function checkKey:
0x0000555555555169 <+0>: endbr64
0x000055555555516d <+4>: push %rbp
0x000055555555516e <+5>: mov %rsp,%rbp
=> 0x0000555555555171 <+8>: sub $0x10,%rsp
0x0000555555555175 <+12>: mov %rdi,-0x8(%rbp)
0x0000555555555179 <+16>: mov -0x8(%rbp),%rax
0x000055555555517d <+20>: lea 0xe80(%rip),%rdx # 0x555555556004
0x0000555555555184 <+27>: mov %rdx,%rsi
0x0000555555555187 <+30>: mov %rax,%rdi
0x000055555555518a <+33>: call 0x555555555070 <strcmp@plt>
0x000055555555518f <+38>: test %eax,%eax
0x0000555555555191 <+40>: jne 0x5555555551a9 <checkKey+64>
0x0000555555555193 <+42>: lea 0xe73(%rip),%rax # 0x55555555600d
0x000055555555519a <+49>: mov %rax,%rdi
0x000055555555519d <+52>: call 0x555555555060 <puts@plt>
0x00005555555551a2 <+57>: mov $0x0,%eax
0x00005555555551a7 <+62>: jmp 0x5555555551bd <checkKey+84>
0x00005555555551a9 <+64>: lea 0xe66(%rip),%rax # 0x555555556016
0x00005555555551b0 <+71>: mov %rax,%rdi
0x00005555555551b3 <+74>: call 0x555555555060 <puts@plt>
0x00005555555551b8 <+79>: mov $0x2,%eax
0x00005555555551bd <+84>: leave
0x00005555555551be <+85>: ret
End of assembler dump.
(gdb) b *0x0000555555555191
Breakpoint 2 at 0x555555555191
(gdb) c
Continuing.
Breakpoint 2, 0x0000555555555191 in checkKey ()
(gdb) i r
rax 0xfffffffa 4294967290
rbx 0x7fffffffe448 140737488348232
rcx 0x74 116
rdx 0x555555556004 93824992239620
rsi 0x555555556004 93824992239620
rdi 0x7fffffffe6fe 140737488348926
rbp 0x7fffffffe300 0x7fffffffe300
rsp 0x7fffffffe2f0 0x7fffffffe2f0
r8 0x0 0
r9 0x7ffff7fca380 140737353917312
r10 0x7fffffffe040 140737488347200
r11 0x203 515
r12 0x2 2
r13 0x0 0
r14 0x555555557db8 93824992247224

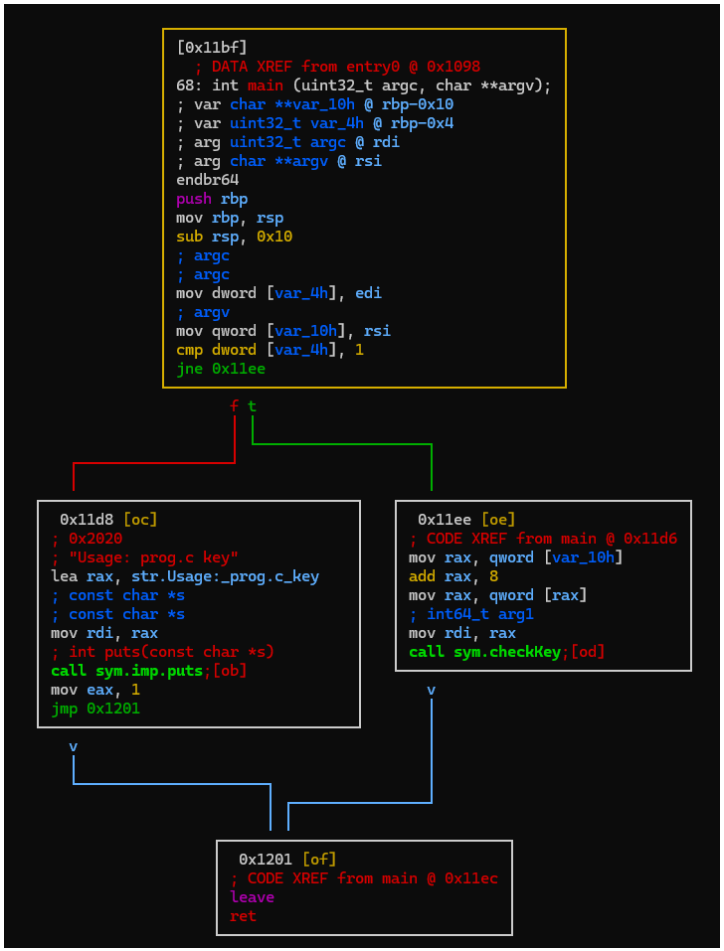
```
r15            0x7ffff7ffd000      140737354125312
rip            0x555555555191      0x555555555191 <checkKey+40>
eflags         0x286               [ PF SF IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
fs_base        0x7ffff7fae740      140737353803584
gs_base        0x0                 0
(gdb) ni
0x00005555555551a9 in checkKey ()
(gdb) set $rip = 0x0000555555555193
(gdb) ni
0x000055555555519a in checkKey ()
(gdb) c
Continuing.
Unlocked
[Inferior 1 (process 2309) exited normally]
(gdb)
```

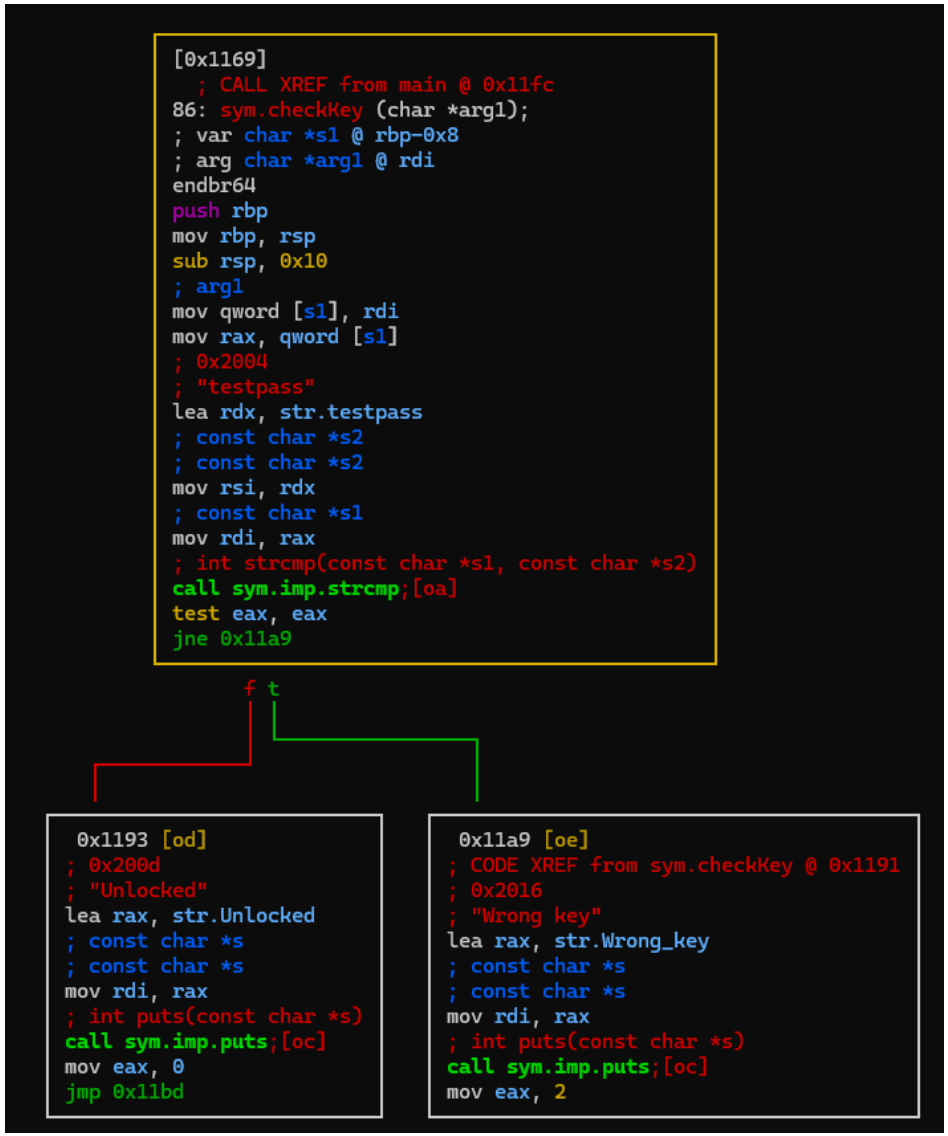
Physical Attack Vector

Even without debugger, a controlled physical shock could theoretically do this.

Radare2

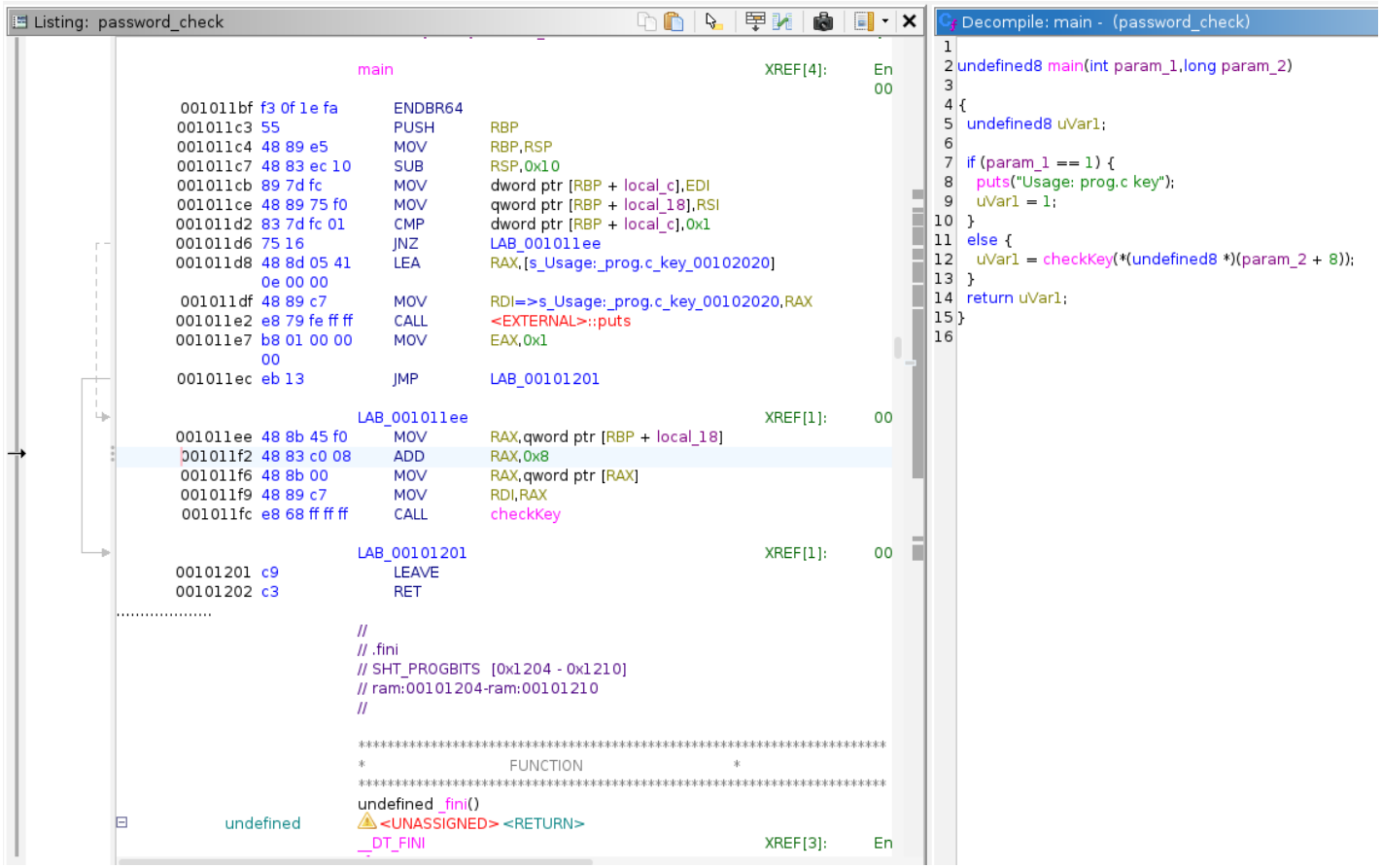
Without running the binary, the assembly instructions can be analyzed.




[0x1169]
; CALL XREF from main @ 0x11fc
86: sym.checkKey (char *arg1);
; var char *s1 @ rbp-0x8
; arg char *arg1 @ rdi
endbr64
push rbp
mov rbp, rsp
sub rsp, 0x10
; arg1
mov qword [s1], rdi
mov rax, qword [s1]
; 0x2004
; "testpass"
lea rdx, str.testpass
; const char *s2
; const char *s2
mov rsi, rdx
; const char *s1
mov rdi, rax
; int strcmp(const char *s1, const char *s2)
call sym.imp.strcmp;[oa]
test eax, eax
jne 0x11a9
f t
0x1193 [od]
; 0x200d
; "Unlocked"
lea rax, str.Unlocked
; const char *s
; const char *s
mov rdi, rax
; int puts(const char *s)
call sym.imp.puts;[oc]
mov eax, 0
jmp 0x11bd
0x11a9 [oe]
; CODE XREF from sym.checkKey @ 0x1191
; 0x2016
; "Wrong key"
lea rax, str.Wrong_key
; const char *s
; const char *s
mov rdi, rax
; int puts(const char *s)
call sym.imp.puts;[oc]
mov eax, 2

```
[0x00001169]> pdda
    ; assembly                                  | /* r2dec pseudo code output (r2 5.5.0) */
                                                | /* password_check @ 0x1169 */
                                                | #include <stdint.h>
                                                |
    ; (fcn) sym.checkKey ()                     | int64_t checkKey (char * arg1) {
                                                |     char * s1;
                                                |     rdi = arg1;
    0x00001169 endbr64                          |
    0x0000116d push rbp                         |
    0x0000116e mov rbp, rsp                     |
    0x00001171 sub rsp, 0x10                    |
    0x00001175 mov qword [rbp - 8], rdi         |     *((rbp - 8)) = rdi;
    0x00001179 mov rax, qword [rbp - 8]         |     rax = *((rbp - 8));
    0x0000117d lea rdx, [rip + 0xe80]           |     rdx = "testpass";
    0x00001184 mov rsi, rdx                     |
    0x00001187 mov rdi, rax                     |
    0x0000118a call 0x1070                      |     eax = strcmp (rax, rdx);
    0x0000118f test eax, eax                    |
                                                |     if (eax == 0) {
    0x00001191 jne 0x11a9                       |
    0x00001193 lea rax, [rip + 0xe73]           |         rax = "Unlocked";
    0x0000119a mov rdi, rax                     |
    0x0000119d call 0x1060                      |         puts (rax);
    0x000011a2 mov eax, 0                       |         eax = 0;
    0x000011a7 jmp 0x11bd                       |
                                                |     } else {
    0x000011a9 lea rax, [rip + 0xe66]           |         rax = "Wrong key";
    0x000011b0 mov rdi, rax                     |
    0x000011b3 call 0x1060                      |         puts (rax);
    0x000011b8 mov eax, 2                       |         eax = 2;
                                                |     }
    0x000011bd leave                            |
    0x000011be ret                              |     return rax;
                                                | }
[0x00001169]>
```

Ghidra


Listing: password_check
main XREF[4]: En
00
001011bf f3 0f 1e fa ENDBR64
001011c3 55 PUSH RBP
001011c4 48 89 e5 MOV RBP,RSP
001011c7 48 83 ec 10 SUB RSP,0x10
001011cb 89 7d fc MOV dword ptr [RBP + local_c],EDI
001011ce 48 89 75 f0 MOV qword ptr [RBP + local_18],RSI
001011d2 83 7d fc 01 CMP dword ptr [RBP + local_c],0x1
001011d6 75 16 JNZ LAB_001011ee
001011d8 48 8d 05 41 0e 00 00 LEA RAX,[s_Usage:_prog.c_key_00102020]
001011df 48 89 c7 MOV RDI=>s_Usage:_prog.c_key_00102020,RAX
001011e2 e8 79 fe ff ff CALL <EXTERNAL>::puts
001011e7 b8 01 00 00 00 MOV EAX,0x1
001011ec eb 13 JMP LAB_00101201
LAB_001011ee XREF[1]: 00
001011ee 48 8b 45 f0 MOV RAX,qword ptr [RBP + local_18]
001011f2 48 83 c0 08 ADD RAX,0x8
001011f6 48 8b 00 MOV RAX,qword ptr [RAX]
001011f9 48 89 c7 MOV RDI,RAX
001011fc e8 68 ff ff ff CALL checkKey
LAB_00101201 XREF[1]: 00
00101201 c9 LEAVE
00101202 c3 RET
//
// .fini
// SHT_PROGBITS [0x1204 - 0x1210]
// ram:00101204-ram:00101210
//
FUNCTION
undefined _fini()
undefined <UNASSIGNED> <RETURN>
__DT_FINI XREF[3]: En
Decompile: main - (password_check)
undefined8 main(int param_1,long param_2)
{
undefined8 uVar1;
if (param_1 == 1) {
puts("Usage: prog.c key");
uVar1 = 1;
}
else {
uVar1 = checkKey(*(undefined8 *)(param_2 + 8));
}
return uVar1;
}

Ghidra

Decompile: main - (password_check)

```
1
2 int main(int argc,char **argv)
3
4 {
5  int ret;
6
7  if (argc == 1) {
8    puts("Usage: prog.c key");
9    ret = 1;
10 }
11 else {
12   ret = checkKey(argv[1]);
13 }
14 return ret;
15 }
16
```

Decompile: checkKey - (password_check)

```
1
2 undefined8 checkKey(char *param_1)
3
4 {
5  int iVar1;
6  undefined8 uVar2;
7
8  iVar1 = strcmp(param_1,"testpass");
9  if (iVar1 == 0) {
10   puts("Unlocked");
11   uVar2 = 0;
12  }
13  else {
14   puts("Wrong key");
15   uVar2 = 2;
16  }
17  return uVar2
18 }
19
```

Decompile: checkKey - (password_check)

```
1
2 int checkKey(char *inputKey)
3
4 {
5  int ret
6
7  ret = strcmp(inputKey,"testpass");
8  if (ret == 0) {
9    puts("Unlocked");
10   ret = 0;
11 }
12 else {
13   puts("Wrong key");
14   ret = 2;
15 }
16 return ret;
17 }
18
```

Side-Channeling: Timing Attack

Target: PIN Program

- `pin_checker`
- from picoCTF 2022, Forensics. Difficulty: Hard
- 8,383KB. Impractical to open with binary analysis tool.
- Complexity means there are probably a lot of operatons, including conditional checks.

```
$ pbjk (demo) > ./pin_checker
Please enter your 8-digit PIN code:
00000000
8
Checking PIN...
Access denied.
$ pbjk (demo) > ./pin_checker
Please enter your 8-digit PIN code:
00000001
8
Checking PIN...
Access denied.
$ pbjk (demo) > ./pin_checker
Please enter your 8-digit PIN code:
0000000000001
14
Incorrect length.
```

Timing Issue

- There is a timing issue with the PIN checking.

```
$ pbjk (demo) > time echo '00000000' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m0.283s
user    0m0.132s
sys     0m0.001s
$ pbjk (demo) > time echo '10000000' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m0.260s
user    0m0.126s
sys     0m0.013s
$ pbjk (demo) > time echo '20000000' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m0.177s
user    0m0.137s
sys     0m0.005s
$ pbjk (demo) > time echo '30000000' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m0.221s
user    0m0.142s
sys     0m0.009s
$ pbjk (demo) > time echo '40000000' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m0.308s
user    0m0.266s
sys     0m0.005s
```

```
real    0m0.550s
user    0m0.522s
sys     0m0.014s
$ pbjk (demo) > time echo '48380000' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m0.574s
user    0m0.519s
sys     0m0.017s
$ pbjk (demo) > time echo '48390000' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m0.806s
user    0m0.779s
sys     0m0.014s
```

```
$ pbjk (demo) > time echo '48390511' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m1.052s
user    0m1.040s
sys     0m0.015s
$ pbjk (demo) > time echo '48390512' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access denied.

real    0m1.044s
user    0m1.022s
sys     0m0.031s
$ pbjk (demo) > time echo '48390513' | ./pin_checker
Please enter your 8-digit PIN code:
8
Checking PIN...
Access granted. You may use your PIN to log into the master server.

real    0m1.131s
user    0m1.150s
sys     0m0.009s
```

Side Channeling: Power Analysis

Target: AES Encryption on Embedded System

- from picoCTF 2023, Cryptography. Difficulty: Hard
- given plaintext, power measurement is given. Goal is to extract encryption key.
- Issue with hamming weights of bytes.

```
$ pbjk (demo) > nc saturn.picoctf.net 60682
Please provide 16 bytes of plaintext encoded as hex: 00000000000000000000000000000000
power measurement result:  [106, 106, 122, 116, 101, 101, 85, 83, 61, 58, 30, 33, 35, 44, 32, 33, 52, 82, 122, 108, 83, 73,
 83, 118, 140, 143, 129, 116, 86, 111, 88, 122, 89, 73, 80, 110, 153, 116, 109, 97, 89, 99, 105, 104, 71, 81, 87, 80, 40, 68
, 84, 77, 97, 90, 109, 123, 89, 99, 51, 95, 106, 108, 103, 75, 97, 76, 72, 61, 67, 94, 127, 129, 106, 92, 106, 119, 118, 121
 84, 81, 62, 62, 50, 59, 106, 113, 107, 83, 80, 88, 63, 70, 74, 91, 73, 61, 71, 86, 87, 88, 86, 87, 59, 81, 65, 93, 86, 137
, 72, 46, 45, 80, 118, 119, 121, 117, 133, 132, 115, 100, 67, 65, 57, 67, 52, 48, 77, 93, 87, 73, 92, 83, 78, 70, 73, 93, 60
, 143, 125, 99, 78, 59, 55, 85, 97, 97, 74, 75, 74, 52, 74, 91, 119, 116, 108, 85, 99, 109, 119, 99, 98, 110, 121, 125, 136,
16, 80, 68, 42, 48, 39, 61, 56, 62, 49, 100, 139, 118, 98, 100, 92, 96, 70, 106, 100, 102, 107, 114, 119, 103, 90, 81, 88, 9
, 162, 172, 192, 218, 205, 197, 166, 198, 195, 181, 154, 158, 175, 189, 200, 191, 158, 160, 156, 153, 133, 158, 193, 192, 18
1, 209, 231, 217, 210, 201, 188, 147, 155, 166, 209, 209, 231, 231, 247, 218, 187, 169, 181, 185, 182, 177, 180, 171, 151, 1
02, 263, 243, 207, 151, 88, 113, 120, 190, 153, 199, 130, 153, 140, 197, 161, 158, 159, 187, 192, 128, 179, 174, 247, 218, 2
6, 193, 192, 147, 162, 200, 195, 189, 163, 148, 141, 128, 139, 73, 146, 167, 215, 209, 256, 231, 158, 100, 126, 196, 208, 20
 170, 114, 158, 158, 183, 173, 169, 167, 172, 200, 195, 154, 92, 140, 199, 261, 241, 249, 229, 222, 246, 181, 174, 171, 252
 156, 177, 156, 171, 148, 169, 161, 171, 160, 162, 182, 197, 191, 180, 186, 215, 210, 197, 160, 143, 129, 153, 175, 186, 175
, 163, 177, 134, 122, 109, 143, 167, 175, 152, 166, 173, 192, 171, 173, 158, 166, 174, 180, 181, 155, 142, 141, 162, 196, 17
, 193, 172, 141, 83, 120, 164, 157, 108, 152, 200, 198, 115, 130, 171, 242, 263, 256, 214, 218, 232, 270, 235, 252, 240, 213
, 183, 202, 228, 176, 115, 57, 79, 144, 206, 243, 228, 197, 200, 198, 172, 194, 213, 193, 120, 132, 217, 247, 208, 159, 207
 205, 154, 135, 161, 128, 114, 121, 197, 237, 232, 213, 183, 168, 159, 190, 179, 220, 176, 151, 143, 163, 246, 234, 241, 221
, 169, 145, 178, 175, 177, 174, 183, 204, 173, 155, 132, 159, 161, 173, 168, 167, 179, 187, 183, 177, 192, 208, 207, 170, 18
2, 185, 194, 205, 182, 198, 194, 209, 180, 201, 200, 233, 206, 175, 173, 162, 206, 184, 207, 147, 140, 107, 175, 166, 165, 1
, 163, 141, 136, 193, 193, 147, 82, 77, 147, 174, 212, 155, 154, 99, 104, 131, 139, 119, 112, 123, 126, 114, 141, 177, 188,
137, 112, 123, 210, 244, 219, 160, 124, 137, 178, 190, 201, 215, 251, 283, 273, 203, 174, 170, 240, 197, 171, 105, 110, 114
 181, 186, 173, 184, 111, 172, 175, 176, 178, 195, 221, 140, 102, 119, 171, 195, 187, 171, 187, 171, 188, 180, 185, 181, 199
, 197, 195, 177, 160, 157, 181, 192, 189, 182, 178, 184, 171, 194, 205, 227, 205, 209, 168, 153, 159, 199, 206, 158, 141, 16
```

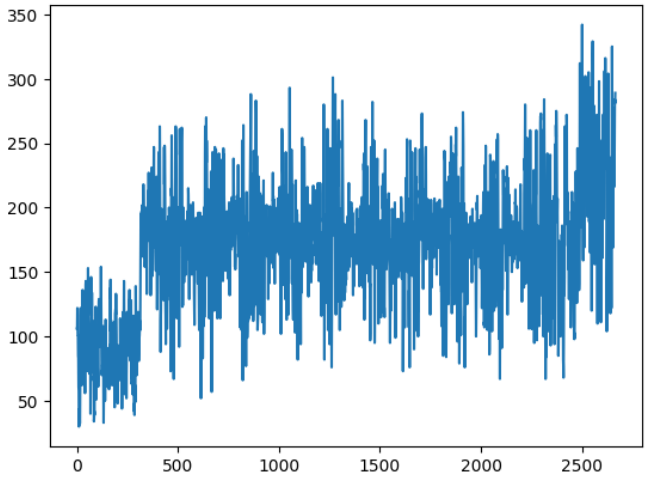


Figure: Plaintext: 0x00, 0x00, $\cdots$, 0x00

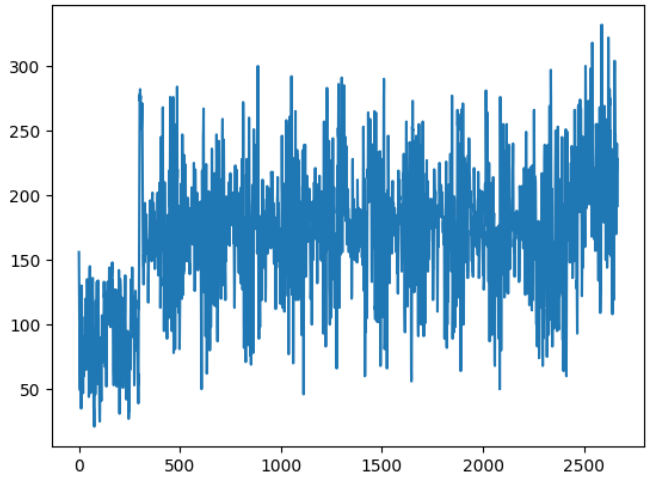


Figure: Plaintext: 0xFF, 0xFF, $\cdots$, 0xFF